TASK 2 – Choose the Data Types:

Recommended types and portability:

Item	Recommended type	Reason / range consideration
Meter reading	unsigned short or uint16_t	The brief specifies a 16-bit register. Confirm USHRT_MAX; uint16_t exists only where an exact 16-bit type is available.
Units consumed	unsigned int / uint32_t	Non-negative; wide enough for the difference between two 16-bit readings.
Household count / loop index	size_t or unsigned long long as appropriate	size_t is natural for array sizes; district-wide counts must not rely on a 16-bit int.
Single bill in paise	long long	Integer arithmetic avoids binary floating-point rounding.
Total collection in paise	long long / int64_t	A district total may exceed a 32-bit signed long.
Tariff rate in paise per unit	int or long	Small integer values, with a type chosen consistently for arithmetic.
Text category	enum or char array	enum is suitable for a fixed set of named categories; string if values come from external data.

C does not guarantee that int, long or long long have the same size on every platform. Use sizeof to measure the local implementation and limits.h / float.h to inspect ranges. A common 64-bit Windows compiler uses 4-byte int and 4-byte long, while long long is 8 bytes; many 64-bit Unix-like systems use an 8-byte long. These are common examples, not universal guarantees.

```
#include <stdio.h>
#include <limits.h>
#include <float.h>

int main(void) {
    printf("sizeof(short) = %zu, USHRT_MAX = %u\n",
           sizeof(short), (unsigned)USHRT_MAX);
    printf("sizeof(int) = %zu, INT_MAX = %d\n",
           sizeof(int), INT_MAX);
    printf("sizeof(long) = %zu, LONG_MAX = %ld\n",
           sizeof(long), LONG_MAX);
    printf("sizeof(long long) = %zu, LLONG_MAX = %lld\n",
           sizeof(long long), LLONG_MAX);
    printf("FLT_DIG = %d, DBL_DIG = %d\n", FLT_DIG,
DBL_DIG);
    return 0;
}
```

Why money is stored as integer paise:

A rupee has 100 paise. A bill such as ₹162.50 is represented as 16,250 paise. Integer addition and multiplication are exact as long as the chosen integer type does not overflow. In contrast, many decimal fractions cannot be represented exactly in binary floating point. Repeatedly adding float rupee amounts can accumulate small representation errors.

For example, a bill of ₹162.50 repeated 4,000 times is exactly ₹650,000.00, or 65,000,000 paise. Store the individual bill as 16,250 paise and sum in a sufficiently wide integer. The integer result is exact

Meter rollover without if–else:

Expected output on a system where unsigned short is 2 bytes and has maximum 65535:
Sizeof (meter_t) = 2 bytes, maximum reading = 65535
Consumption: 330 units
Consumption: 256 units

The second result is 256 because the 16-bit register wraps modulo 65,536. The difference is 120 − 65,400 = −65,280; interpreted modulo 65,536, this is 256. The cast to unsigned short produces the register-width result. This approach assumes exactly one rollover and the stated register width.

Coding:

```
#include <stdio.h>
#include <limits.h>

typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
   /* Conversion back to meter_t applies modulo 65536 on a
 16-bit register. */
   return (unsigned short)(current - previous);
}

int main(void)
{
   printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
         sizeof(meter_t), (unsigned)USHRT_MAX);
   printf("Consumption: %u units\n", consumption(1200, 1530));
   printf("Consumption: %u units\n", consumption(65400, 120));
   return 0;
}
```

Alternative: double rupees:

An alternative is to store amounts as double rupees and round at the billing boundary, for example to two decimal places. Addition remains O(1), just as integer addition does. A double usually occupies 8 bytes, while a long long commonly occupies 8 bytes too. However, binary floating-point cannot represent every decimal amount exactly, and rounding policy must be explicit. Integer paise is preferable when every bill must be exact to the paisa.

When n bills are added, each floating-point operation may introduce a small rounding error. The total error depends on the values and order of operations; it is not safe to assume it will always cancel. Integer paise has zero representation error until overflow occurs.